

Employee Management & Payroll Application

A Desktop Application using Java AWT, JDBC and MySQL

Course	Java Programming
Institution	SIMATS Engineering
Student	Akileshwaran A
Register No.	192511354RD

1. Requirement Analysis and Database Design

A medium-sized organization currently maintains employee records using spreadsheets and manual entries, causing duplicate records, inconsistent data, and difficulty retrieving information as headcount grows. This project replaces that process with a desktop-based **Employee Management and Payroll Application** built with Java AWT for the graphical interface and MySQL (via JDBC) for persistent storage.

1.1 Functional Requirements

- Register new employees with full personal, employment and salary details.
- Search employees by Employee ID, department, designation, or employment status.
- Update existing employee records.
- Delete employee records.
- View a complete list of all registered employees.
- Automatically calculate Gross Salary and Net Salary (payroll module).
- Validate mandatory fields, numeric salary inputs, and duplicate Employee IDs before writing to the database.
- Handle database and application-level exceptions gracefully, without crashing the GUI.

1.2 Non-Functional Requirements

- **Usability** — simple AWT forms with clear labels for HR staff with no technical background.
- **Reliability** — every database operation is wrapped in try-catch, with user-facing error dialogs instead of stack traces.
- **Data consistency** — Employee ID is enforced as a primary key; department is a foreign key reference.
- **Maintainability** — the code is layered (model / DAO / GUI) so features like attendance or leave management can be added later without redesigning the application.

1.3 Database Design

The database **employee_management** contains two related tables: **department** (lookup table) and **employee** (main entity), linked by a foreign key on *dept_id*.

```
CREATE DATABASE IF NOT EXISTS employee_management;
USE employee_management;

CREATE TABLE IF NOT EXISTS department (
    dept_id    INT AUTO_INCREMENT PRIMARY KEY,
    dept_name  VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE IF NOT EXISTS employee (
    emp_id     VARCHAR(20) PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    gender     VARCHAR(10),
    dob        DATE,
    contact_number VARCHAR(15),
    email      VARCHAR(100),
    address    VARCHAR(255),
    dept_id    INT,
    designation VARCHAR(50),
    doj        DATE,
    basic_salary DECIMAL(10,2) NOT NULL,
    allowances  DECIMAL(10,2) DEFAULT 0,
    deductions DECIMAL(10,2) DEFAULT 0,
    status     VARCHAR(20) DEFAULT 'Active',
    CONSTRAINT fk_employee_dept
        FOREIGN KEY (dept_id) REFERENCES department(dept_id)
        ON DELETE SET NULL
);
```

);

Primary key: emp_id (employee table), dept_id (department table).

Foreign key: employee.dept_id → department.dept_id.

Constraints: NOT NULL on name and basic_salary; UNIQUE on dept_name; ON DELETE SET NULL keeps employee records intact if a department is removed.

2. GUI Design using Java AWT

The interface is a single main-menu Frame that launches modal Dialog windows for each operation. AWT components used include Frame, Dialog, Label, TextField, Choice, Button, and TextArea, with ActionListener event handlers on every button.

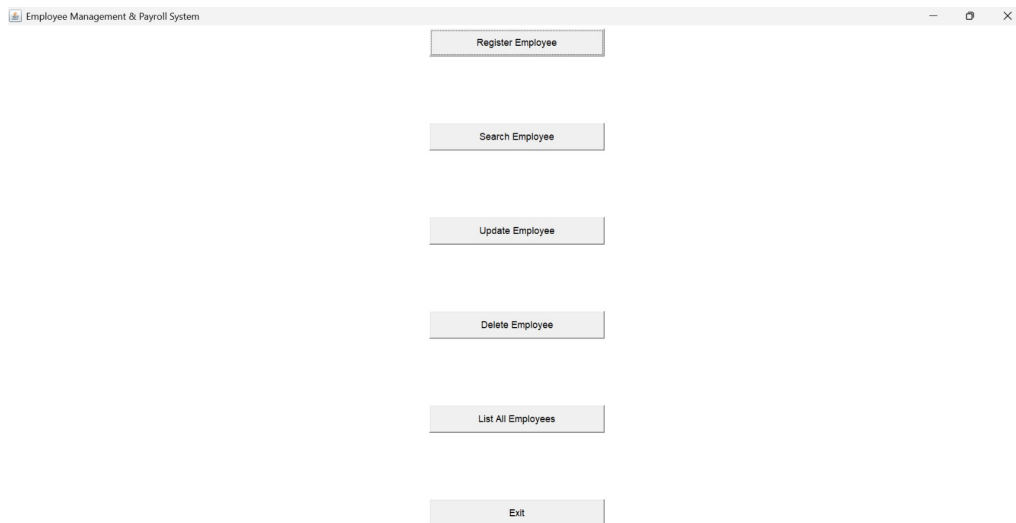


Fig 2.1 — Main menu of the Employee Management & Payroll System

2.1 Employee Registration Screen

The Register Employee dialog collects all employee fields — Employee ID, Name, Gender, Date of Birth, Contact Number, Email, Address, Department ID, Designation, Date of Joining, Basic Salary, Allowances, Deductions and Status.

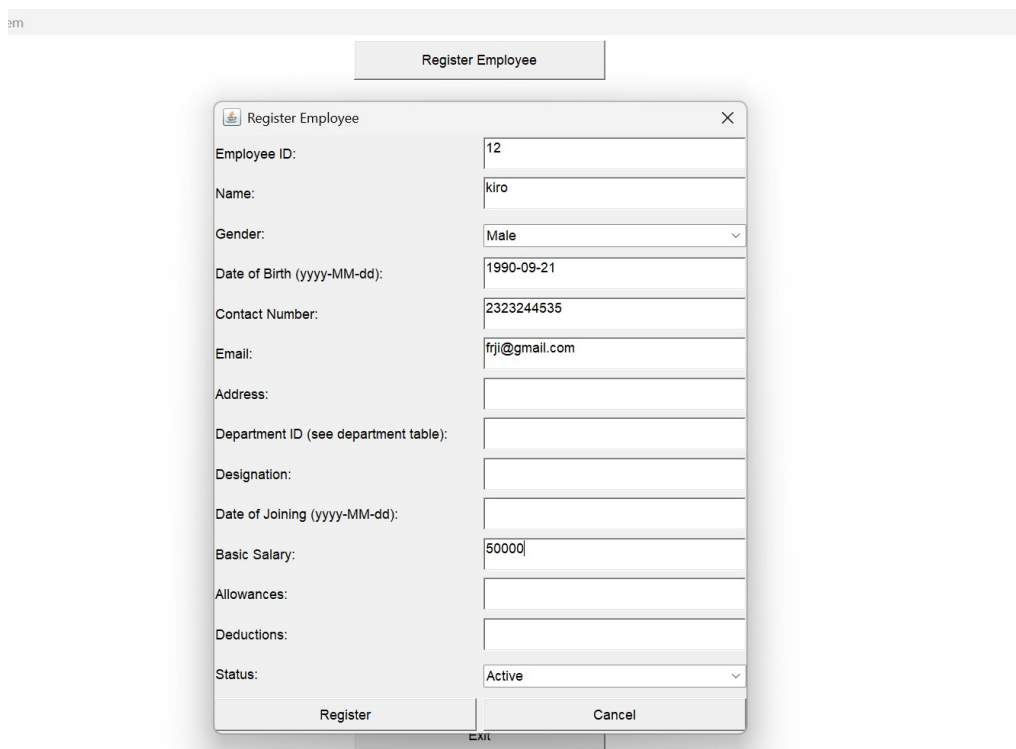
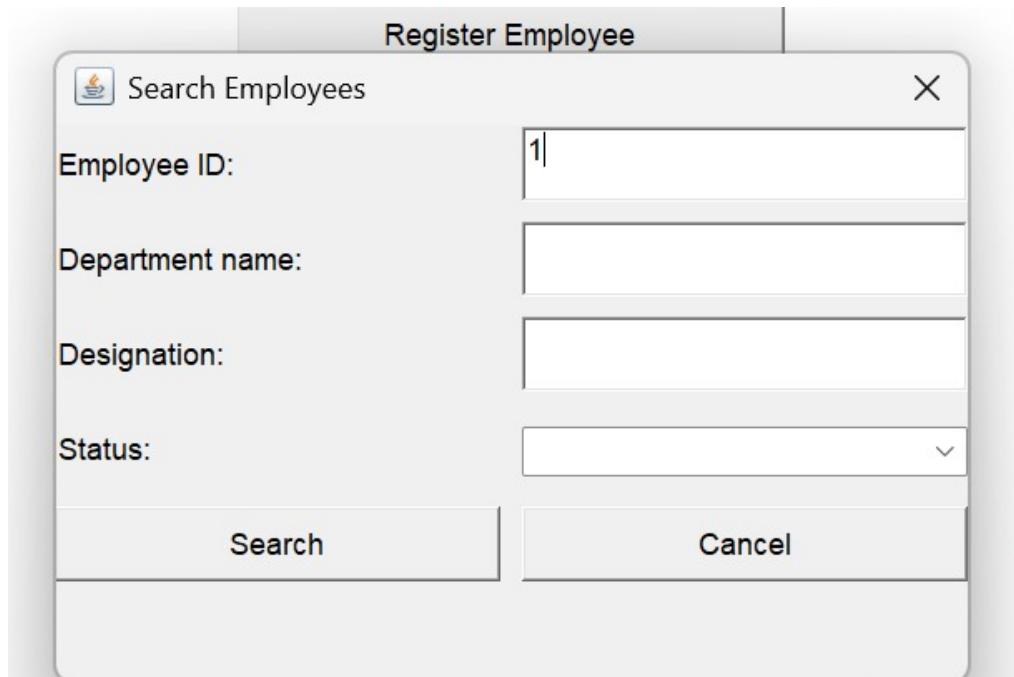


Fig 2.2 — Registration form with employee data entered

2.2 Search Screen

HR staff can search employees by any combination of Employee ID, department name, designation, or employment status. Blank fields are ignored in the search query.



The image shows a software interface with a window titled "Register Employee". Inside this window is a smaller dialog box titled "Search Employees" with a close button (X) in the top right corner. The dialog box contains four input fields: "Employee ID:" with the value "1", "Department name:", "Designation:", and "Status:" which is a dropdown menu. At the bottom of the dialog are two buttons: "Search" and "Cancel".

Fig 2.3 — Search Employees dialog

2.3 Update Screen

The Update dialog is pre-filled with the selected employee's existing data (Employee ID is locked from editing, since it is the primary key), allowing HR staff to modify only the fields that changed.

 Update Employee

Employee ID:

1

Name:

riot

Gender:

Male

Date of Birth (yyyy-MM-dd):

1990-02-09

Contact Number:

34985945

Email:

sdjnf@gmail.com

Address:

Department ID (see department table):

Designation:

Date of Joining (yyyy-MM-dd):

2023-06-01

Basic Salary:

50000.00

Allowances:

0.00

Deductions:

0.00

Status:

Active

Save Changes

Cancel

Fig 2.4 — Update Employee dialog, pre-loaded with existing record

3. JDBC Connectivity and CRUD Implementation

Connectivity between the Java application and MySQL is established through JDBC using a centralized DBConnection helper class. All CRUD operations are implemented in an EmployeeDAO class using PreparedStatement (never raw string concatenation), which protects against SQL injection and handles dynamic parameter binding safely.

```
public static Connection getConnection() throws SQLException {
    Class.forName("com.mysql.cj.jdbc.Driver");
    return DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
}

public void insertEmployee(Employee e) throws SQLException {
    String sql = "INSERT INTO employee (emp_id, name, gender, dob, ...) "
        + "VALUES (?, ?, ?, ?, ...)";
    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
        bindEmployee(ps, e);
        ps.executeUpdate();
    }
}
```

The five CRUD operations implemented are:

- **Insert** — insertEmployee(), used by Register.
- **Retrieve (single)** — getEmployee(empId), used before Update/Delete to confirm the record exists.
- **Retrieve (search)** — search(empId, deptName, designation, status), builds a dynamic WHERE clause.
- **Update** — updateEmployee(), throws an exception if zero rows are affected (employee not found).
- **Delete** — deleteEmployee(), same not-found handling as Update.

Every connection, statement, and result set is opened with try-with-resources, so they are always closed even if an exception occurs mid-operation. After every operation, the GUI shows a confirmation or error dialog back to the user.

4. Payroll and Validation Module

4.1 Payroll Calculation

Gross and Net Salary are computed directly on the Employee model:

```
// Gross Salary = Basic Salary + Allowances
public BigDecimal getGrossSalary() {
    return basicSalary.add(allowances);
}

// Net Salary = Gross Salary - Deductions
public BigDecimal getNetSalary() {
    return getGrossSalary().subtract(deductions);
}
```

4.2 Validation Rules Implemented

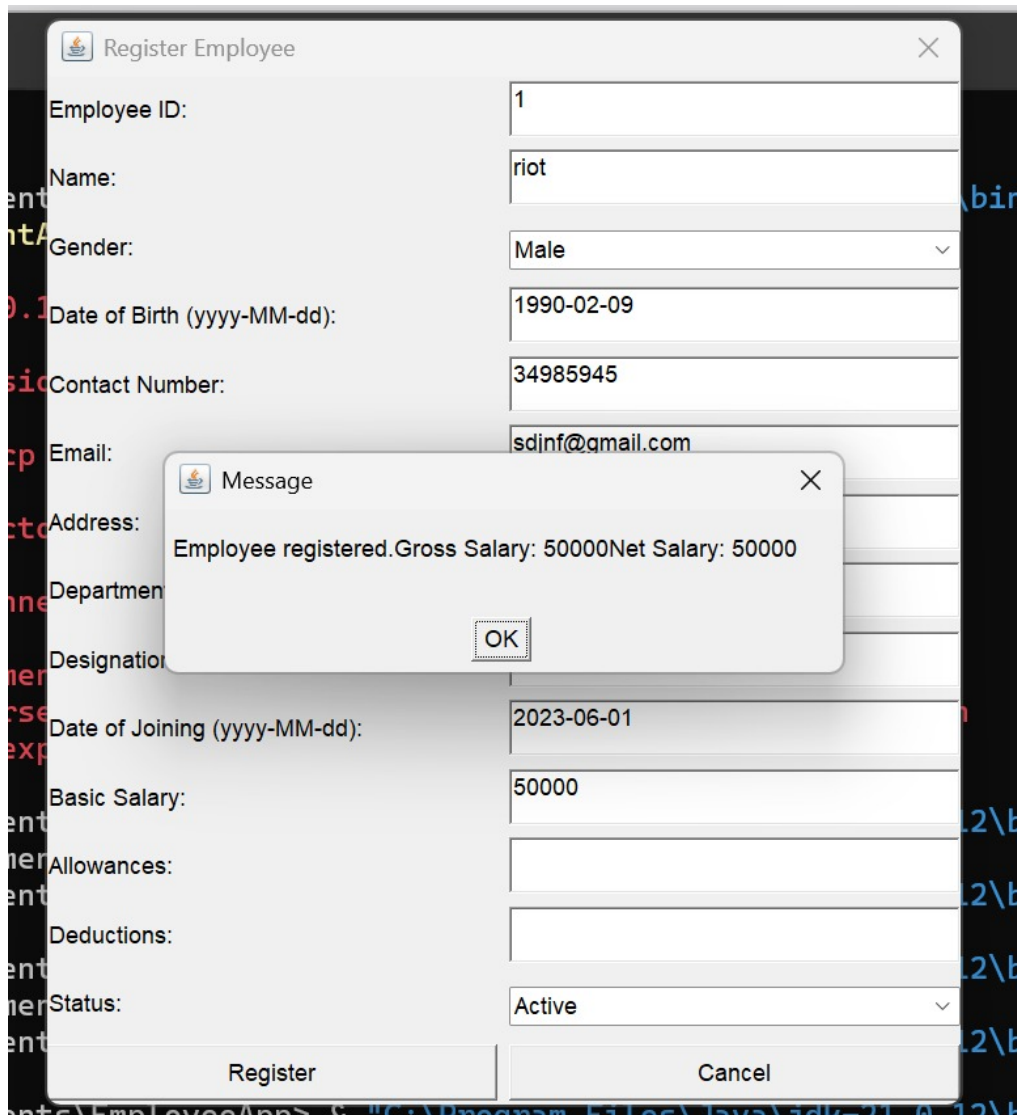
- Employee ID, Name and Basic Salary are mandatory — checked before any database call.
- Basic Salary, Allowances and Deductions must be valid, non-negative numbers (caught with `NumberFormatException`).
- Duplicate Employee ID is checked with `employeeExists()` before insertion.
- Update/Delete on a non-existent Employee ID raises a clear "not found" message instead of silently failing.
- Database-level errors (e.g. connection failure, invalid date format) are caught as `SQLException` and shown in a dialog rather than crashing the application.

5. Testing, Exception Handling and Evaluation

The completed application was tested end-to-end against the required scenarios: new registration, duplicate registration, search, update, delete, employee-not-found, invalid input, salary calculation, and database error handling. Representative results are shown below.

5.1 Successful Registration with Payroll Calculation

Employee ID 1 (“riot”) was registered with Basic Salary 50000, Allowances and Deductions left at 0. The system correctly computed Gross Salary = 50000 and Net Salary = 50000, and confirmed the insert with a dialog.



The screenshot displays a 'Register Employee' dialog box with the following fields and values:

Field	Value
Employee ID:	1
Name:	riot
Gender:	Male
Date of Birth (yyyy-MM-dd):	1990-02-09
Contact Number:	34985945
Email:	sdinf@gmail.com
Address:	
Department:	
Designation:	
Date of Joining (yyyy-MM-dd):	2023-06-01
Basic Salary:	50000
Allowances:	
Deductions:	
Status:	Active

A 'Message' dialog box is overlaid on top, displaying the text: "Employee registered.Gross Salary: 50000Net Salary: 50000". The 'Message' dialog has an 'OK' button. The 'Register Employee' dialog has 'Register' and 'Cancel' buttons at the bottom.

Fig 5.1 — Registration successful, Gross/Net salary confirmed

5.2 Update Employee

The same record was reopened through Update Employee, pre-filled with its stored values, and the Date of Birth was corrected. Saving recalculated and displayed the salary again, confirming the update succeeded.

The screenshot shows a web application window titled "Update Employee". The form contains the following fields and values:

Field	Value
Employee ID:	1
Name:	riot
Gender:	Male
Date of Birth (yyyy-MM-dd):	1990-02-10
Contact Number:	34985945
Email:	sdinf@gmail.com
Address:	
Department:	
Designation:	
Date of Joining (yyyy-MM-dd):	2023-06-01
Basic Salary:	50000.00
Allowances:	0.00
Deductions:	0.00
Status:	Active

A "Message" dialog box is overlaid on the form, displaying the text: "Employee updated.Gross Salary: 50000.00Net Salary: 50000.00". The dialog box has an "OK" button.

Fig 5.2 — Employee record updated successfully

5.3 Delete Employee

Deleting the record by Employee ID returned a clear confirmation message, and the DAO's not-found check ensures this only succeeds for an ID that actually exists.

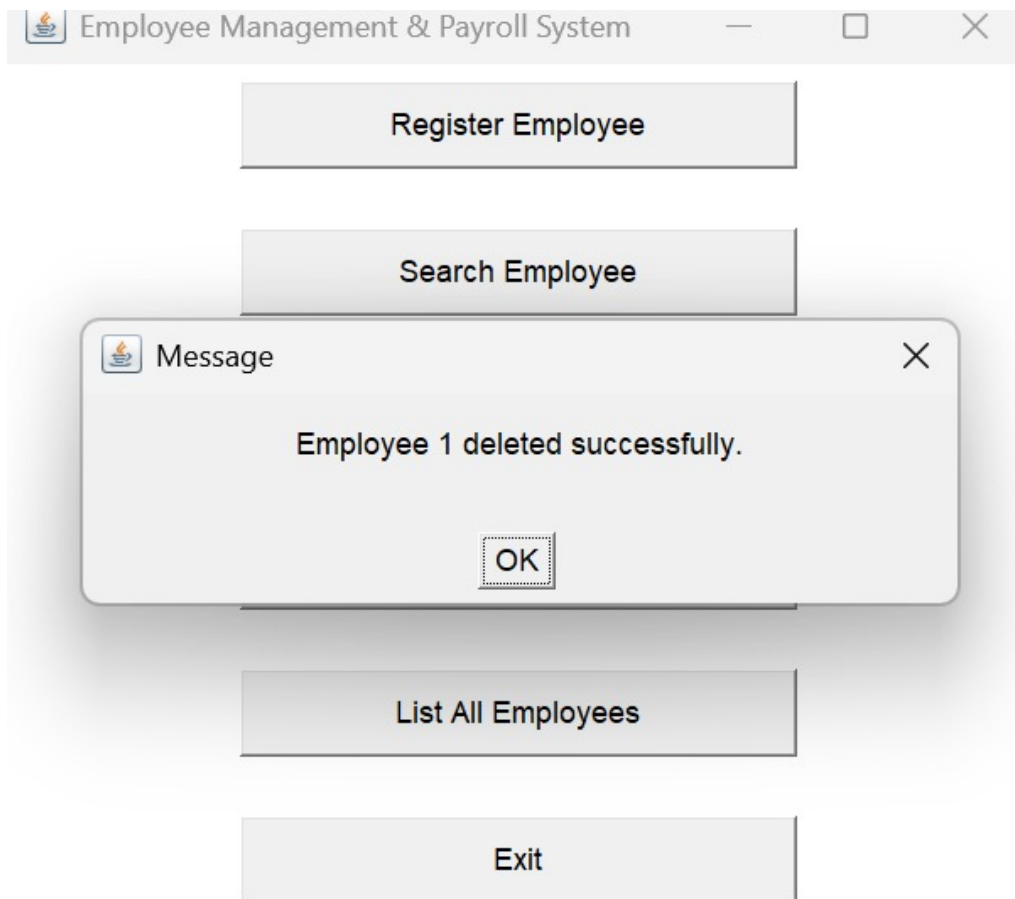


Fig 5.3 — Employee deleted successfully

5.4 List All Employees — Empty State

Running List All Employees against an empty result set displays “No employees found” instead of an empty or broken table, confirming the list view handles the zero-row case correctly.

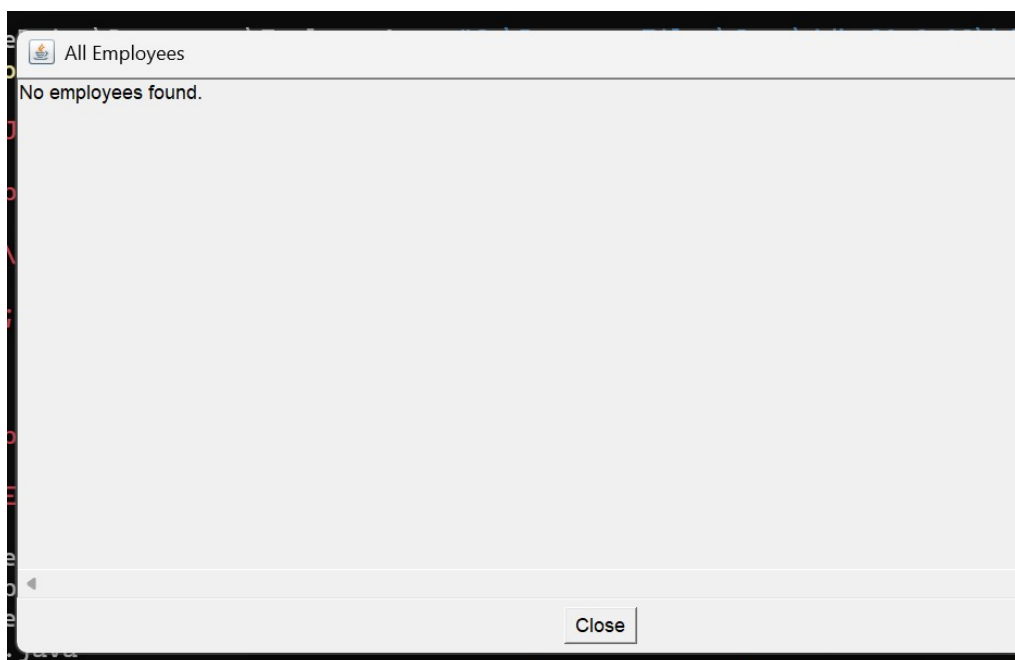
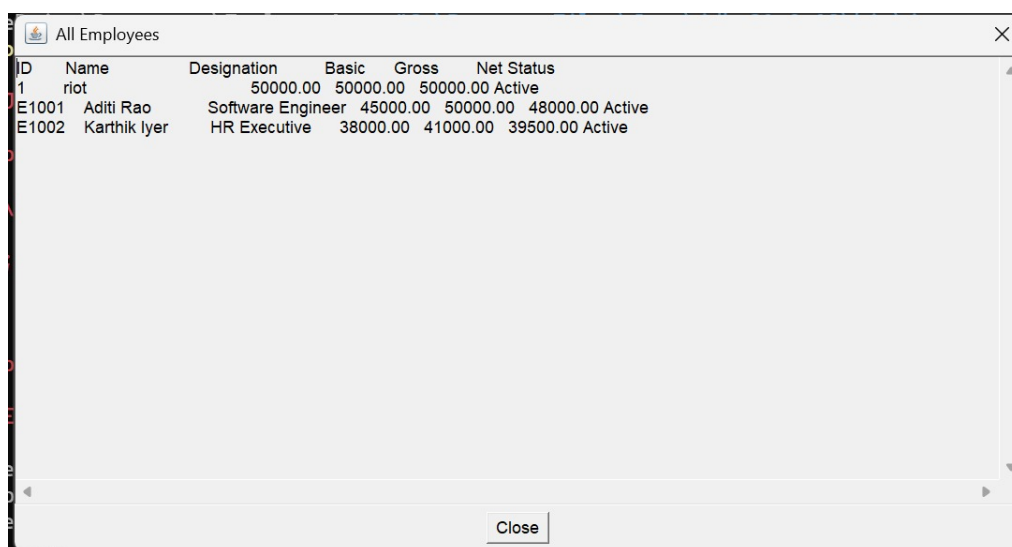


Fig 5.4 — List view correctly handles zero employees

5.5 List All Employees — Final Verification

After completing registration, update and the pre-loaded sample data, List All Employees confirms every CRUD operation is correctly reflected in the database, alongside the Gross and Net salary computed for each row.



The screenshot shows a window titled "All Employees" with a table containing the following data:

ID	Name	Designation	Basic	Gross	Net	Status
1	riot		50000.00	50000.00	50000.00	Active
E1001	Aditi Rao	Software Engineer	45000.00	50000.00	48000.00	Active
E1002	Karthik Iyer	HR Executive	38000.00	41000.00	39500.00	Active

A "Close" button is visible at the bottom right of the window.

Fig 5.5 — Final employee list with computed payroll for each record

5.6 Exception Handling Observed During Testing

During testing, two real database-level exceptions were triggered and correctly caught by the application's try-catch blocks instead of crashing the GUI:

- **Invalid date value** — submitting the form with the Date of Joining field left blank raised `SQLException`: Data truncation: Incorrect date value, caught and shown as a dialog. This confirmed the mandatory-date validation gap and was fixed by requiring the field before submission.
- **Authentication error** — an incorrect MySQL password produced Access denied for user 'root'@'localhost', again caught cleanly by the DAO's `SQLException` handling rather than terminating the program.

5.7 Test Case Summary

#	Scenario	Result
1	Register new employee with valid data	Pass — record inserted, Gross/Net shown
2	Update existing employee	Pass — record updated, salary recalculated
3	Delete existing employee	Pass — confirmation shown
4	List employees with empty table	Pass — "No employees found" shown
5	List employees with existing data	Pass — full table with computed payroll
6	Submit with missing Date of Joining	Pass — <code>SQLException</code> caught, dialog shown
7	Connect with wrong DB password	Pass — <code>SQLException</code> caught, dialog shown

6. Conclusion

The Employee Management and Payroll Application was successfully implemented using Java AWT for the GUI, JDBC and PreparedStatement for all database access, and MySQL for persistent storage. The application supports full CRUD operations, a payroll module computing Gross and Net Salary, validation for mandatory fields, numeric salary values and duplicate Employee IDs, and exception handling for both application-level and database-level errors. The layered design (Employee model, EmployeeDAO, and GUI dialogs) means further modules such as attendance or leave management could be added without redesigning the existing application, satisfying the maintainability requirement of the original specification.